Yongjae Kwon 권용재

Backend Developer

EMAIL **yongjae116@gmail.com** | PHONE **+82 10-9470-1704** | LOCATION **Yongin, South Korea**
GITHUB **github.com/YongjaeKwon** | PORTFOLIO **www.yongjaekwon.com**

SUMMARY

A Java/Spring backend developer who turns complex field operations into systems users can run without interruption. I replaced paper-based school inspections with a flow that connects scheduling, checklists, signatures, and re-inspection history, and moved long-running bulk attachment jobs to async processing so users can watch progress and download results. I align with operations staff on the real task order, then design the UI, server, and DB together — owning testing, deployment, and operations after release.

SKILLS

Backend		Java, Spring Boot, Spring MVC, Spring Security, MyBatis
Database		Oracle, MariaDB, MySQL
Frontend	WORK	JavaScript, Vue, HTML, CSS, JSP, WebSquare5, jQuery
	PROJECTS	TypeScript, React, Next.js, Redux Toolkit, Pinia
Infra · Delivery		Linux, Tomcat, Jenkins, Gradle, Maven, Git, SVN

EXPERIENCE

TG Narae LLC — Web Developer · Solutions Team

Jun 2024 – Present · Full-time

B2B Partner Portal (PPS)

Feb 2025 – Present

A B2B portal where headquarters and about 500 partner companies handle partner registration and contracts, plus training, certification, documentation, and evaluation for 824 field engineers (CEs). I coordinate requirements with operations and business staff, analyze existing data flows, and own DB modeling, HTTP API and SQL design and development, external-system integrations, user acceptance, and Jenkins deployment and operations.

Tech | Java 21 · Spring Boot · MyBatis · MariaDB · Oracle · Vue

PPS Highlights

Moving bulk attachment downloads to async processing

Problem	Collecting and zipping attachments for 300–400 engineers in a single request held the screen for minutes, with no way to tell whether the job was running or had failed.
Decision	The real issue was the request and the job being tied into one flow, not compression speed — so I split the job out and exposed its status.
Implementation	A Spring async job handles compression; the request API returns a job ID and a status API reports running / done / failed. Job state lives in a Hazelcast distributed map, so both servers report the same progress and the job survives a refresh.
Result	Freed HTTP requests from long-running zip jobs and gave users a flow where they track progress and download only finished results.

Unifying the split headquarters account provisioning process

Problem	Accounts had to be created separately in the internal system and the portal, with the portal ID entered directly into the DB — risking omissions and mismatched records.
Decision	Entering data into two systems separately could not prevent omissions, so the portal became the single provisioning gateway.
Implementation	Combined ID creation, organization validation, initial credential issuance, and the internal-system integration into one procedure; if the integration fails, the portal save rolls back in the same transaction.
Result	Collapsed the three-step process — internal account, portal account, manual DB entry — into a single screen, and added inventory and service-ownership checks before account changes to block wrongful deactivations.

PPS Highlights · continued

Rate-limiting password-reset codes across two servers

Problem	Password reset sends a verification code by notification message, but nothing stopped repeated clicks or automated requests — and with two servers, per-server in-memory limits fall apart once requests are load-balanced.
Decision	The limit state had to be shared between servers, and identifying values such as account and phone number must not be stored raw as limiter keys.
Implementation	Stored TTL cooldowns in a Hazelcast distributed map with putIfAbsent-based atomic acquisition, so only one of any concurrent requests sends a code. Limiter keys are SHA-256 hashes of account + phone number, environments without Hazelcast fall back to a local map, and codes come from SecureRandom.
Result	The same limit applies whichever server receives the request, and users see the remaining wait time. Boundary conditions — acquisition right after expiry, concurrent races — are pinned by 30 related unit tests.

Centralizing notification-message policy

Problem	Recipients and send conditions were written into each feature's source, so a policy change meant editing several code paths.
Decision	Separate the frequently changing rules from the shared send logic, so policy changes no longer require a deployment.
Implementation	Recipients and send conditions moved to DB-managed configuration; a shared send service looks up the policy for each message type. Key conditions and exception paths are pinned by unit tests.
Result	Five send points across four business services now go through one shared service; a policy change is a DB configuration update verified by four unit tests.

Education Device Management System (TSMS)

Sep 2025 – Present

Manages about 110,000 education devices for the Seoul Metropolitan Office of Education's one-device-per-student program — registration through delivery, installation, repairs, and inspections, with full history. I align business rules and reference data with operations staff and own DB modeling, HTTP API and SQL design, screen and server development, external integrations, user acceptance, and deployment.

Tech | WebSquare5 · JavaScript · JSP · Java 11 · Spring MVC · MyBatis · MariaDB

TSMS Highlights

Removing exposed API keys and unifying external call paths

Problem	25 screens called map, address, and education-administration APIs directly, exposing keys in the browser and requiring per-screen edits whenever integration details changed.
Decision	Rotating the keys would not stop the exposure or the repeated edits from recurring — the external-call responsibility had to move from the screens to the server.
Implementation	Consolidated external calls into a shared server-side API layer, with keys and endpoint URLs held in server configuration. The affected screens were switched to one common path and response shape.
Result	Eliminated browser key exposure across 25 screens and reduced the scattered call and configuration change points to a single server path.

Re-keying live inspection data and cleaning duplicates in production

Problem	Inspection targets were stored by a 14-digit input serial while full serials have 15 digits, so ambiguous matches could map wrongly or store duplicates — and with the system already live, fixing the screens alone would not clean the accumulated data.
Decision	Re-key storage to the full 15-digit serial, and clean the existing data while preserving inspection results — with a procedure that could be rolled back before it touched production.
Implementation	Switched storage to 15-digit serials, resolving 14-digit input through candidate search confirmed by school assignment. Wrote de-duplication SQL prioritized by grade, class, number, and inspection history, and applied it to the production DB together with backup, rollback, and post-verification SQL.
Result	Removed duplicate rows while preserving inspection results and details, and ID-based updates keep the problem from recurring. (applied to production, Jul 2026)

TSMS Highlights · continued

Bulk device registration checks and QR issuance

Problem	Bulk spreadsheet uploads could include serial numbers missing from production records, or devices that were already registered.
Decision	Move validation ahead of the save — checking reference data and duplicates before registration instead of cleaning up afterward.
Implementation	Serial numbers are checked against production records, and already-registered devices return their current school and user. QR codes carry an encrypted token instead of internal identifiers.
Result	Applied to the QR issuance flow for 108,237 of 111,593 devices, blocking wrong registrations and identifier exposure up front. (as of Jul 2026)

Fixing a consolidated-view lookup that could not finish in 60 seconds

Problem	Repair-history and settlement-detail lookups went through a consolidated multi-table view, and as data grew the screens degraded to waiting tens of seconds.
Decision	The execution plan showed the customer filter never reached inside the view — the whole view was materialized and then filtered (about 7.7 billion estimated rows) — so the query structure had to change, not the indexes.
Implementation	Removed the view and rewrote the lookups as direct base-table joins over only the columns the screens actually use, so the customer condition applies through indexes from the start.
Result	Re-measured on production data (Sep 2026, heaviest customer): the old query could not finish within a 60-second cap, while the rewrite returns the same rows in 77–124 ms — at least a 770x improvement; plan estimate down from ~7.7 billion to 1,852 rows.

PERSONAL PROJECTS

ticket-rush · first-come ticketing reservation system

Aug 2026 – Present · Personal

A reservation system built for zero double-bookings under concurrent load. Each seat is guarded by three layers — Redis SET NX holds, domain rules rejecting expired holds at payment, and a DB unique constraint on (show, seat) — with a Redis-outage scenario reproduced in integration tests to prove exactly one confirmation survives. Implements idempotent processing, a transactional outbox, and SSE seat-status streams; a contention test with 100 concurrent requests for one seat yields exactly one success and zero double bookings. 65 automated tests.

Tech | Java 21 · Spring Boot 4 · Spring Modulith · MySQL · Redis · Flyway · Testcontainers · GitHub Actions

ReachRich · personal investment research & operations platform

Mar 2026 – Present · Personal

Rebuilt a personal investment platform into six modules — account tracking on the Toss Securities Open API, KRX data collection, strategy validation, paper trading, and a dashboard. Implemented date-keyed idempotent persistence with partial-failure isolation, FastAPI query APIs, a React UI, and GitHub Actions health checks with Telegram alerts; 205 backend tests cover the data collection, validation, and operation logic.

Tech | Python · FastAPI · SQLAlchemy · SQLite · Parquet · React · TypeScript · GitHub Actions

EDUCATION

Training	**Samsung SW Academy For Youth (SSAFY), 8th cohort** · **Web development program**	Jul 2022 – Jun 2023
Education	**Ajou University** · **B.S. in e-Business (transfer)**	Mar 2018 – Aug 2020
Education	**California State University, Chico** · **Business Administration, attended before transfer**	Jan 2014 – May 2015
Certification	**SQLD (SQL Developer)** · **Korea Data Agency**	Sep 2024